Q1

Sunday, December 15, 2024 12:03 AM

1. Add Logging to Monitor Ticket Operations

Solution for Logger modification:

```
package logging;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;
import java.util.UUID;

public class Logger {

   private static final String LOG_FILE = "resources/ticket_operations.log";

   public static void logTicketOperation(String ticketId, String operationType) {
      String logEntry = String.format("%s | %s | %s | %s",
         LocalDateTime.now(),
         operationType,
         ticketId,
         "Ticket Operation Logged"
      );

      // Console logging
      System.out.println(logEntry);

      // File logging
      try (BufferedWriter writer = new BufferedWriter(new FileWriter(LOG_FILE, true))) {
         writer.write(logEntry);
         writer.newLine();
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
}
```

Modification for TicketPool:

```
package core;

import java.util.Collections;
import java.util.LinkedList;
```

```
import java.util.List;
import logging.Logger;

public class TicketPool implements TicketOperation {
   private final List<String> tickets = Collections.synchronizedList(new LinkedList<>());

   @Override
   public synchronized void addTickets(String ticket) {
      tickets.add(ticket);
      Logger.logTicketOperation(ticket, "ADD");
   }

   @Override
   public synchronized String removeTicket() {
      if (!tickets.isEmpty()) {
         String ticket = tickets.remove(0);
         Logger.logTicketOperation(ticket, "REMOVE");
         return ticket;
      }
      return null;
   }

   public int getTicketCount() {
      return tickets.size();
   }
}
```

why is there a buffered writer and filer writer

ChatGPT said:

ChatGPT

The combination of **BufferedWriter** and **FileWriter** is used to balance **performance** and **functionality** when writing to files. Here's why both are used:

1. What is FileWriter?

FileWriter is a low-level class used to write characters directly to a file.

- **Purpose**:
  - It handles the actual task of opening the file and writing characters to it.
  - The constructor allows specifying whether to overwrite the file or append to it.
- **Drawback**:
  - Writing directly to a file character-by-character can be inefficient because each write operation might trigger a direct interaction with the disk.

2. What is BufferedWriter?

BufferedWriter is a high-level class that wraps around a Writer (like FileWriter) to improve

performance by adding buffering.

- **Purpose**:
  - It adds a buffer (a temporary memory storage) between the program and the file.
  - Instead of writing to the file every time a single character or line is written, it accumulates data in memory and writes it in chunks.
  - This reduces the number of disk I/O operations, which are slow compared to memory operations.
- **Advantage**:
  - Significant performance boost when writing many small pieces of data (e.g., logging messages).
  - Methods like newLine() and write() are optimized to work with buffered data.

3. Why Use Both?

The **BufferedWriter** is used for performance, while the **FileWriter** is used for file handling. Together, they create an efficient system for writing data to files.

Combined Workflow:

1. **FileWriter**:
   - Opens the file (log.txt) and establishes the connection for writing.
   - Handles the appending mode (true) or overwriting mode (false).
2. **BufferedWriter**:
   - Adds a memory buffer to optimize file-writing operations.
   - Reduces direct writes to the disk by batching multiple writes together.

4. Without BufferedWriter

If you only use FileWriter:

java
Copy code

```
try (FileWriter writer = new FileWriter("log.txt", true)) {
   writer.write("This is a log message.");
   writer.write(System.lineSeparator());
} catch (IOException e) {
   e.printStackTrace();
}
```

- Each write() call interacts directly with the file. If there are frequent small writes (e.g., logging), this would result in many costly disk I/O operations.

5. With Both (BufferedWriter and FileWriter)

When you use both:

java
Copy code

```
try (BufferedWriter writer = new BufferedWriter(new FileWriter("log.txt", true))) {
   writer.write("This is a log message.");
   writer.newLine();
} catch (IOException e) {
   e.printStackTrace();
}
```

- Data is first written to the buffer in memory.
- When the buffer is full, or the stream is closed, the data is written to the file in a single, efficient operation.

6. Summary of Their Roles

Feature	FileWriter	BufferedWriter
Purpose	Writes characters to a file.	Adds buffering for performance.
Performance	Slower, direct file access.	Faster, writes in chunks.
Usage	Opens the file and appends/overwrites.	Provides efficient write operations.
Interaction	Works directly with the disk.	Works with memory before the disk.

Using **BufferedWriter** with **FileWriter** combines the functionality of file handling and the efficiency of buffered writing, making it an ideal choice for logging or frequent file operations.

he Logger class is a utility designed to log messages to both the console and a file. Let's break it down step by step:

1. Class Declaration

java
Copy code
public class Logger {

- The Logger class is declared as public, meaning it can be accessed from any other class in the program.

2. Constant Declaration

java
Copy code
private static final String log_file = "log.txt";

- **private**: The log_file variable is accessible only within the Logger class.
- **static**: The variable is associated with the class rather than any specific instance. All instances of the Logger class share the same log_file value.
- **final**: The value of log_file cannot be changed once initialized.
- **Value**: "log.txt" is the name of the file where logs will be stored.

3. Log Method

java
Copy code
public static void log(String message) {

- **public**: The log method can be called from any class.
- **static**: The method belongs to the class and can be invoked without creating an instance of Logger.
- **Parameter**: The method takes a single string parameter message, which is the content to be logged.

4. Timestamped Message

java
Copy code
String timeStampedMessage = LocalDateTime.now() + ": " + message;

- **LocalDateTime.now()**: Gets the current date and time when the method is called.
- **Concatenation**: Combines the timestamp with the message. For example:

  bash
  Copy code
  2024-12-15T15:30:00: This is a log message.

5. Console Logging

java
Copy code
System.out.println(timeStampedMessage);

- The System.out.println statement outputs the timeStampedMessage to the console for immediate feedback.

6. File Logging

java
Copy code
```
try (BufferedWriter writer = new BufferedWriter(new FileWriter(log_file, true))) {
   writer.write(timeStampedMessage);
   writer.newLine();
} catch (IOException e) {
   e.printStackTrace();
}
```
Explanation:

1. **try-with-resources**:
   - The BufferedWriter is used inside a try-with-resources block, ensuring the writer is automatically closed after the operation.
   - **new FileWriter(log_file, true)**:
     - log_file: Specifies the file to write to (log.txt).
     - true: Enables *append mode*, meaning new messages are added at the end of the file without overwriting existing content.
   - **BufferedWriter**:
     - Buffers the output, improving performance by reducing direct disk writes.
2. **Writing the Message**:
   - writer.write(timeStampedMessage): Writes the timestamped message to the file.
   - writer.newLine(): Adds a newline character to separate entries in the log.
3. **Exception Handling**:
   - If an IOException occurs (e.g., file not found or access issues), it is caught in the catch block.
   - e.printStackTrace() prints the stack trace of the error to help with debugging.

Key Features:

- **Thread Safety**: The log method is not thread-safe by itself, meaning issues might arise in

a multi-threaded environment unless external synchronization is used.

- **Dual Output**: Messages are logged to both the console and a file.
- **Timestamped Logs**: Ensures each log entry has a timestamp for easy tracking.
- **Persistent Logging**: Logs are written to a file, creating a permanent record.

Example Usage:

java
Copy code

```
public class Main {
   public static void main(String[] args) {
      Logger.log("Application started.");
      Logger.log("Processing user request...");
      Logger.log("Application stopped.");
   }
}
```

Output on Console:

makefile
Copy code

```
2024-12-15T15:30:00: Application started.
2024-12-15T15:30:01: Processing user request...
2024-12-15T15:30:02: Application stopped.
```

Contents of log.txt:

makefile
Copy code

```
2024-12-15T15:30:00: Application started.
2024-12-15T15:30:01: Processing user request...
2024-12-15T15:30:02: Application stopped.
```

q2

Sunday, December 15, 2024 12:04 AM

Extending Customer Functionality with Interfaces

```
// Ticket Retrieval Strategy Interface
public interface TicketRetrievalStrategy {

  //method
   String retrieveTicket(TicketPool ticketPool);
}

// Priority Retrieval Strategy
public class PriorityRetrieval implements TicketRetrievalStrategy {
   @Override
   public String retrieveTicket(TicketPool ticketPool) {
      // Implement priority-based ticket retrieval logic
      return ticketPool.removeTicket();
   }
}

// ID Retrieval Strategy
public class IDRetrieval implements TicketRetrievalStrategy {
   private String targetTicketId;

   public IDRetrieval(String targetTicketId) {
      this.targetTicketId = targetTicketId;
   }

   @Override
   public String retrieveTicket(TicketPool ticketPool) {
      // Implement ID-based ticket retrieval logic
      // This is a simplified example and would need more complex implementation
      return targetTicketId;
   }
}

// Modified Customer Class
public class Customer extends AbstractTicketHandler implements Runnable {
   private TicketRetrievalStrategy retrievalStrategy;

   public Customer(TicketPool ticketPool, TicketRetrievalStrategy strategy) {
      super(ticketPool);
      this.retrievalStrategy = strategy;
   }

   @Override
   public void run() {
      String ticket = retrievalStrategy.retrieveTicket(ticketPool);
      if (ticket != null) {
         Logger.log("Customer retrieved: " + ticket);
      }
   }
}
```

```
// TicketPool.java
package org.example.cwfromscratch.Core;

import org.example.cwfromscratch.Logger.Logging;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedList;
import java.util.List;

public class TicketPool implements TicketOperation {
   private final static List<String> tickets =
Collections.synchronizedList(new LinkedList<>());

   @Override
   public void addTicket(String ticket) {
      tickets.add(ticket);
      Logging.Logger(ticket, "added");
   }

   @Override
   public String buyTicket() {
      String ticket = tickets.isEmpty() ? null : tickets.remove(0);
      Logging.Logger(ticket, "remove");
      return ticket;
   }

   // New method to find and remove a specific ticket by ID
   public synchronized String buyTicketById(long targetId) {
      for (int i = 0; i < tickets.size(); i++) {
         String ticket = tickets.get(i);
         String[] parts = ticket.split(":");
         if (parts.length == 2) {
            try {
               long ticketId = Long.parseLong(parts[1]);
               if (ticketId == targetId) {
                  String foundTicket = tickets.remove(i);
                  Logging.Logger(foundTicket, "remove by id");
                  return foundTicket;
               }
            } catch (NumberFormatException e) {
               continue;
            }
         }
      }
      return null;
   }
}

// SpecificIdTicketRetrieval.java
package org.example.cwfromscratch.Thread;

import org.example.cwfromscratch.Core.TicketPool;

public class SpecificIdTicketRetrieval implements
TicketRetrievalStrategy {
   private final long targetId;

   public SpecificIdTicketRetrieval(long targetId) {
      this.targetId = targetId;
   }

   @Override
   public String retrievalStrategy(TicketPool ticketPool) {
      if (!(ticketPool instanceof TicketPool)) {
         return null;
      }
      return ((TicketPool) ticketPool).buyTicketById(targetId);
   }
}
```

Q3

Sunday, December 15, 2024 12:04 AM

3. Exception Handling for Invalid Configurations

```
// Custom Exception
public class InvalidConfigurationException extends Exception {
   public InvalidConfigurationException(String message) {
      super(message);
   }
}

// Modified Configuration Class
public class Configuration {
   // Existing fields...

   public void validate() throws InvalidConfigurationException {
      if (totalTickets <= 0) {
         throw new InvalidConfigurationException("Total tickets must be positive");
      }
      if (ticketReleaseRate <= 0) {
         throw new InvalidConfigurationException("Ticket release rate must be positive");
      }
      if (customerRetrievalRate <= 0) {
         throw new InvalidConfigurationException("Customer retrieval rate must be positive");
      }
      if (maxTicketCapacity <= 0) {
         throw new InvalidConfigurationException("Max ticket capacity must be positive");
      }
   }
}

// Modified CLI
public class CommandLineInterface {
   public static Configuration configureSystem() {
      Scanner scanner = new Scanner(System.in);
      Configuration config;

      while (true) {
         try {
            // Existing input gathering code...
            config = new Configuration(totalTickets, ticketReleaseRate,
                                       customerRetrievalRate, maxTicketCapacity);
            config.validate();
            return config;
         } catch (InvalidConfigurationException e) {
            System.out.println("Invalid Configuration: " + e.getMessage());
            System.out.println("Please re-enter the configuration.");
         }
      }
   }
}
```

q4

Sunday, December 15, 2024 12:05 AM

4. Enhancing Vendor Behavior Using Polymorphism

```
// Abstract Vendor Class
public abstract class AbstractVendor extends AbstractTicketHandler implements Runnable {
    protected int ticketReleaseRate;

    public AbstractVendor(TicketPool ticketPool, int ticketReleaseRate) {
        super(ticketPool);
        this.ticketReleaseRate = ticketReleaseRate;
    }

    @Override
    public abstract void run();
}

// Fast Vendor
public class FastVendor extends AbstractVendor {
    public FastVendor(TicketPool ticketPool, int ticketReleaseRate) {
        super(ticketPool, ticketReleaseRate * 2);
    }

    @Override
    public void run() {
        for (int i = 0; i < ticketReleaseRate; i++) {
            String ticket = "FastTicket-" + System.nanoTime();
            ticketPool.addTickets(ticket);
            Logger.log("Fast Vendor added: " + ticket);

            try {
                Thread.sleep(250); // Faster release
            } catch (InterruptedException e) {
                Logger.log("Fast Vendor interrupted.");
            }
        }
    }
}

// Slow Vendor
public class SlowVendor extends AbstractVendor {
    public SlowVendor(TicketPool ticketPool, int ticketReleaseRate) {
        super(ticketPool, ticketReleaseRate / 2);
    }

    @Override
    public void run() {
        for (int i = 0; i < ticketReleaseRate; i++) {
            String ticket = "SlowTicket-" + System.nanoTime();
            ticketPool.addTickets(ticket);
            Logger.log("Slow Vendor added: " + ticket);
```

```
            try {
                Thread.sleep(1000); // Slower release
            } catch (InterruptedException e) {
                Logger.log("Slow Vendor interrupted.");
            }
        }
    }
}
```

Q5

Sunday, December 15, 2024 12:05 AM

5. Periodic Ticket Statistics Reporting

```
public class TicketStatisticsReporter implements Runnable {
    private TicketPool ticketPool;
    private volatile boolean running = true;

    public TicketStatisticsReporter(TicketPool ticketPool) {
        this.ticketPool = ticketPool;
    }

    @Override
    public void run() {
        while (running) {
            reportStatistics();
            try {
                Thread.sleep(5000); // 5 seconds
            } catch (InterruptedException e) {
                running = false;
            }
        }
    }

    private void reportStatistics() {
        // This would require tracking total tickets added/removed
        // A potential implementation might involve modifying TicketPool
        Logger.log("Ticket Pool Statistics:");
        Logger.log("Current Tickets: " + ticketPool.getTicketCount());
    }

    public void stop() {
        running = false;
    }
}
```

Q6

Sunday, December 15, 2024 12:05 AM

6. GUI for Dynamic Ticket Pool Visualization

```
// In JavaFXInterface.java
private ListView<String> ticketPoolListView;

@Override
public void start(Stage primaryStage) {
    // Existing setup code...

    ticketPoolListView = new ListView<>();
    gridPane.add(new Label("Current Ticket Pool:"), 0, 8);
    gridPane.add(ticketPoolListView, 0, 9, 2, 1);
}

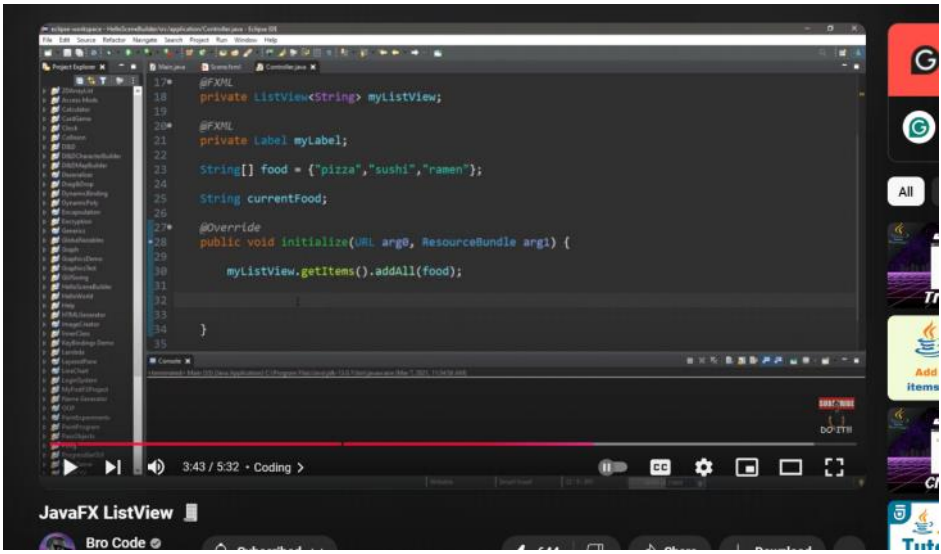
private void updateTicketPoolView(String ticket, boolean isAdded) {
    Platform.runLater(() -> {
        if (isAdded) {
            ticketPoolListView.getItems().add(ticket);
        } else {
            ticketPoolListView.getItems().remove(ticket);
        }
    });
}

// Modify TicketPool to call this method
public class TicketPool implements TicketOperation {
    private JavaFXInterface guiInterface; // Reference to GUI

    public void setGUIInterface(JavaFXInterface gui) {
        this.guiInterface = gui;
    }

    @Override
    public synchronized void addTickets(String ticket) {
        tickets.add(ticket);
        if (guiInterface != null) {
            guiInterface.updateTicketPoolView(ticket, true);
        }
        Logger.logTicketOperation(ticket, "ADD");
    }

    @Override
    public synchronized String removeTicket() {
        if (!tickets.isEmpty()) {
            String ticket = tickets.remove(0);
            if (guiInterface != null) {
                guiInterface.updateTicketPoolView(ticket, false);
            }
            Logger.logTicketOperation(ticket, "REMOVE");
            return ticket;
        }
        return null;
    }
}
```



abstract vs interface

Sunday, December 22, 2024 10:37 AM

Abstraction in Java

Abstraction is one of the core principles of Object-Oriented Programming (OOP). It focuses on hiding the internal implementation details of a class and showing only the essential features to the outside world. This allows you to work at a higher level of logic without worrying about complex underlying details.
In Java, abstraction is achieved using:

1. **Abstract Classes**
2. **Interfaces**

Why Abstraction?

1. **Simplifies Complexity**: By hiding unnecessary implementation details, abstraction makes the system easier to understand.
2. **Focus on Functionality**: Developers can focus on the "what" rather than the "how."
3. **Promotes Reusability**: Allows you to define templates that different implementations can follow.

Abstract Classes

An abstract class:

- Is a class declared using the abstract keyword.
- Can have abstract methods (methods without implementation) and concrete methods (methods with implementation).
- Cannot be instantiated.

Example:

java
Copy code

```
abstract class Vehicle {
   abstract void startEngine(); // Abstract method: no implementation

   void fuel() { // Concrete method: has implementation
      System.out.println("Filling up fuel tank.");
   }
}
class Car extends Vehicle {
   @Override
   void startEngine() {
      System.out.println("Car engine starts with a key.");
   }
}
class ElectricCar extends Vehicle {
   @Override
   void startEngine() {
      System.out.println("Electric car engine starts with a button.");
   }
}
public class Main {
   public static void main(String[] args) {
      Vehicle myCar = new Car();
```

```
        myCar.startEngine(); // Output: Car engine starts with a key.
        myCar.fuel();       // Output: Filling up fuel tank.

        Vehicle myElectricCar = new ElectricCar();
        myElectricCar.startEngine(); // Output: Electric car engine starts with a button.
    }
}
```

Interfaces

An interface:

- Is a fully abstract class (before Java 8) and uses the interface keyword.
- Can have abstract methods, default methods (from Java 8), and static methods.
- Supports multiple inheritance.

Example:

```
java
Copy code
interface Animal {
    void eat(); // Abstract method

    default void sleep() { // Default method
        System.out.println("Sleeping...");
    }
}
class Dog implements Animal {
    @Override
    public void eat() {
        System.out.println("Dog eats bones.");
    }
}
class Cat implements Animal {
    @Override
    public void eat() {
        System.out.println("Cat eats fish.");
    }
}
public class Main {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        myDog.eat();   // Output: Dog eats bones.
        myDog.sleep(); // Output: Sleeping...

        Animal myCat = new Cat();
        myCat.eat();   // Output: Cat eats fish.
        myCat.sleep(); // Output: Sleeping...
    }
}
```

Differences Between Abstract Classes and Interfaces

Feature	Abstract Class	Interface
Implementation	Can have both abstract and concrete methods.	Methods are abstract by default (before Java 8).

Inheritance	Supports single inheritance.	Supports multiple inheritance.
Variables	Can have instance variables.	Variables are public static final by default.
Constructors	Can have constructors.	Cannot have constructors.
Access Modifiers	Methods can have any access level.	Methods are public by default.

Benefits of Abstraction

1. **Security**: Hides implementation details from users.
2. **Maintainability**: Makes code easier to update or modify.
3. **Reusability**: Abstract classes and interfaces define reusable templates for developers.
4. **Flexibility**: Implementation details can vary for different classes using the same abstraction.

Would you like examples of combining abstraction with other OOP concepts, such as polymorphism?

Conclusion

- Use **abstract classes** for creating a base class in a hierarchy where you can share common functionality.
- Use **interfaces** to define capabilities or roles that can be shared across unrelated classes.

Key Points to Consider

1. **Use Abstract Classes When**:
    - You want to share code among related classes (common base class).
    - You need to define protected or private members.
    - You need constructors or instance variables.
2. **Use Interfaces When**:
    - You need to define a **contract** that multiple unrelated classes can implement.
    - You need multiple inheritance (Java allows a class to implement multiple interfaces).
    - You want to add functionality to a class without affecting its existing inheritance.

my AbstractTicketHandler has handletickets abstract method but in abstract vendor doesnt implement but rather it implements in vendor

ChatGPT
This is a perfectly valid design. Since AbstractVendor extends AbstractTicketHandler, it **inherits the abstract handleTickets method** but does not need to implement it unless it is a concrete class. Subclasses of AbstractVendor (such as Vendor) will then be responsible for implementing handleTickets.

Here's how it works:

How Abstract Inheritance Works

1. **AbstractTicketHandler**:
   - Declares an abstract method handleTickets, meaning any concrete class inheriting from it must implement this method.
2. **AbstractVendor**:
   - Extends AbstractTicketHandler.
   - Inherits the abstract handleTickets method but doesn't implement it because it is also an abstract class. This is allowed because abstract classes can defer the implementation of abstract methods to their subclasses.
3. **Vendor** (or similar concrete classes):
   - Extends AbstractVendor.
   - Must implement the handleTickets method to become a concrete (instantiable) class.

Why You Don't Need to Implement handleTickets in AbstractVendor

An abstract class like AbstractVendor acts as a **middle layer** in the inheritance hierarchy. It's perfectly acceptable to leave abstract methods from a superclass (like AbstractTicketHandler) unimplemented in AbstractVendor, as long as the ultimate concrete subclasses (e.g., Vendor) provide the implementation.

- **Abstract Classes**: These act as templates for other classes, containing some functionality that is not yet implemented. They cannot be instantiated directly but can be extended by other classes that implement the missing functionality.
- **File Reader Example**: The video uses an example of an abstract file reader class to demonstrate how abstract classes can simplify code. This class includes basic operations for interacting with files and a key algorithm for reading file contents, which can be extended by subclasses to handle specific parsing needs.
- **Code Reusability**: By placing common algorithms in an abstract class, you can create subclasses that only need to implement specific methods, making the code easier to read and extend.

These points highlight how abstract classes help in managing code complexity and improving reusability in Java programs.

From <https://www.linkedin.com/learning/java-object-oriented-programming-2/using-interfaces-in-java-to-provide-conformity?resume=false&u=76664938>

answers

Monday, December 30, 2024 10:14 AM

How to Answer the Sample Paper
The following is a **sample demonstration** of how you are expected to structure your answers in the provided text area. This example is not a complete answer but serves to illustrate the **format and approach** you should follow when writing your responses.
1. Logging for Ticket Operations
Task: Modify the Logger class to log all ticket operations (addTickets and removeTicket) in a standardized format.
Solution:

- Update the Logger class to include the ticket ID, operation type (add/remove), and timestamp for each log entry.
- In the TicketPool class, use the updated Logger to log operations inside the addTickets and removeTicketmethods.
- **Test Case**:
    1. Add tickets using the addTickets method and ensure logs include ticket IDs and timestamps.
    2. Remove tickets using the removeTicket method and verify logs for consistency.

1. Logging for Ticket Operations

Logger.java

```
package logging;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;

public class Logger {
    private static final String LOG_FILE = "resources/logs.txt";

    public static void log(String ticketId, String operation) {
        String timeStampedMessage = String.format("%s: Ticket ID: %s, Operation: %s",
                LocalDateTime.now(), ticketId, operation);
        System.out.println(timeStampedMessage);
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(LOG_FILE, true))) {
            writer.write(timeStampedMessage);
            writer.newLine();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

TicketPool.java

```
package core;

import logging.Logger;
import java.util.Collections;
import java.util.LinkedList;
import java.util.List;

public class TicketPool implements TicketOperation {
    private final List<String> tickets = Collections.synchronizedList(new LinkedList<>());
```

```
    @Override
    public synchronized void addTickets(String ticket) {
        tickets.add(ticket);
        Logger.log(ticket, "add");
    }

    @Override
    public synchronized String removeTicket() {
        String ticket = tickets.isEmpty() ? null : tickets.remove(0);
        Logger.log(ticket, "remove");
        return ticket;
    }

    public int getTicketCount() {
        return tickets.size();
    }
}
```

Test Code

Logger Test (Task 1: Logging for Ticket Operations)

```
import core.TicketPool;

public class LoggerTest {
    public static void main(String[] args) {
        TicketPool ticketPool = new TicketPool();

        // Add tickets and verify logs
        ticketPool.addTickets("Ticket-1");
        ticketPool.addTickets("Ticket-2");

        // Remove tickets and verify logs
        ticketPool.removeTicket();
        ticketPool.removeTicket();
    }
}
```

- **2. Extending Customer Functionality with Interfaces**
  Task: Introduce TicketRetrievalStrategy for different retrieval methods.
  Solution:
- Define an interface TicketRetrievalStrategy with methods for priority-based and ID-based retrieval.
- Implement PriorityRetrieval and IDRetrieval classes.
- Update Customer class to use the TicketRetrievalStrategy interface.
- **Test Case**: Use both retrieval methods in different scenarios and confirm expected behavior.
- # 2. Extending Customer Functionality with Interfaces
  TicketRetrievalStrategy.java
- ```
  package core;

  public interface TicketRetrievalStrategy {
      String retrieveTicket();
  }
  ```
- **PriorityRetrieval.java**
- ```
  package core;

  public class PriorityRetrieval implements TicketRetrievalStrategy {
      private final TicketPool ticketPool;
  ```

```
    public PriorityRetrieval(TicketPool ticketPool) {
        this.ticketPool = ticketPool;
    }

    @Override
    public String retrieveTicket() {
        // Implement logic for priority-based retrieval (simplified here) - YOU NEED TO
ADD THE LOGIC
        return ticketPool.removeTicket(); // Placeholder
    }
}
```

- **IDRetrieval.java**
- package core;

```
public class IDRetrieval implements TicketRetrievalStrategy {
    private final TicketPool ticketPool;

    public IDRetrieval(TicketPool ticketPool) {
        this.ticketPool = ticketPool;
    }

    @Override
    public String retrieveTicket() {
        // Implement logic for ID-based retrieval (simplified here) - YOU NEED TO ADD
THE LOGIC
        return ticketPool.removeTicket(); // Placeholder
    }
}
```

- **Customer.java**
- package threads;

```
import core.TicketRetrievalStrategy;
import logging.Logger;

public class Customer implements Runnable {
    private final TicketRetrievalStrategy retrievalStrategy;

    public Customer(TicketRetrievalStrategy retrievalStrategy) {
        this.retrievalStrategy = retrievalStrategy;
    }

    @Override
    public void run() {
        while (true) {
            String ticket = retrievalStrategy.retrieveTicket();
            if (ticket != null) {
                Logger.log(ticket, "retrieved");
            } else {
                Logger.log("No tickets available", "retrieve");
                break;
            }
        }
    }
}
```

- **Customer Functionality Test (Task 2: Extending Customer Functionality with Interfaces)**
- import core.*;

```
import threads.Customer;

public class CustomerFunctionalityTest {
    public static void main(String[] args) {
        TicketPool ticketPool = new TicketPool();

        // Add tickets to pool
        ticketPool.addTickets("Priority-Ticket-1");
        ticketPool.addTickets("ID-Ticket-2");

        // Test Priority Retrieval
        TicketRetrievalStrategy priorityRetrieval = new PriorityRetrieval(ticketPool);
        Thread priorityCustomer = new Thread(new Customer(priorityRetrieval));

        // Test ID Retrieval
        TicketRetrievalStrategy idRetrieval = new IDRetrieval(ticketPool);
        Thread idCustomer = new Thread(new Customer(idRetrieval));

        priorityCustomer.start();
        idCustomer.start();
    }
}
```

- **3. Exception Handling for Invalid Configurations**
  Task: Create a custom InvalidConfigurationException to validate system parameters.
  Solution:
- Add validation checks in the Configuration class to ensure positive integers for all parameters.
- Throw InvalidConfigurationException with descriptive messages for invalid values.
- **Test Case**: Provide invalid inputs via CLI and verify exception handling prompts the user for corrections.

- # 3. Exception Handling for Invalid Configurations
  InvalidConfigurationException.java
- ```
  package exceptions;

  public class InvalidConfigurationException extends Exception {
      public InvalidConfigurationException(String message) {
          super(message);
      }
  }
  ```
- **Configuration.java**
- ```
  package config;

  import exceptions.InvalidConfigurationException;

  public class Configuration {
      private int totalTickets;
      private int ticketReleaseRate;
      private int customerRetrievalRate;
      private int maxTicketCapacity;

      public Configuration(int totalTickets, int ticketReleaseRate, int customerRetrievalRate,
  int maxTicketCapacity)
              throws InvalidConfigurationException {
          if (totalTickets <= 0 || ticketReleaseRate <= 0 || customerRetrievalRate <= 0 ||
  maxTicketCapacity <= 0) {
              throw new InvalidConfigurationException("All configuration values must be
  positive.");
  ```

```
        }
        this.totalTickets = totalTickets;
        this.ticketReleaseRate = ticketReleaseRate;
        this.customerRetrievalRate = customerRetrievalRate;
        this.maxTicketCapacity = maxTicketCapacity;
    }
}
```

- **Exception Handling Test (Task 3: Invalid Configurations)**
-
```
import config.Configuration;
import exceptions.InvalidConfigurationException;

public class ConfigurationTest {
    public static void main(String[] args) {
        try {
            // Valid configuration
            Configuration validConfig = new Configuration(100, 10, 5, 50);
            System.out.println("Configuration created successfully!");

            // Invalid configuration
            Configuration invalidConfig = new Configuration(-100, 10, 5, 50);
        } catch (InvalidConfigurationException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

- **4. Enhancing Vendor Behavior Using Polymorphism**
  Task: Add FastVendor and SlowVendor classes with different ticket release rates.
  Solution:
- Create FastVendor (double rate) and SlowVendor (half rate) classes inheriting from Vendor.
- Use polymorphism to seamlessly integrate these into the system.
- **Test Case**: Run both vendor types concurrently and ensure their rates differ as expected.
- # 4. Enhancing Vendor Behavior Using Polymorphism
  FastVendor.java
-
```
package threads;

import core.TicketPool;
import logging.Logger;

public class FastVendor extends Vendor {
    public FastVendor(TicketPool ticketPool, int ticketReleaseRate) {
        super(ticketPool, ticketReleaseRate * 2);
    }

    @Override
    public void run() {
        super.run();
        Logger.log("FastVendor completed", "info");
    }
}
```

- **SlowVendor.java**
-
```
package threads;

import core.TicketPool;
import logging.Logger;

public class SlowVendor extends Vendor {
```

```
    public SlowVendor(TicketPool ticketPool, int ticketReleaseRate) {
        super(ticketPool, ticketReleaseRate / 2);
    }

    @Override
    public void run() {
        super.run();
        Logger.log("SlowVendor completed", "info");
    }
}
```

- **Vendor Behavior Test (Task 4: Enhancing Vendor Behavior)**
- ```
  import core.TicketPool;
  import threads.FastVendor;
  import threads.SlowVendor;

  public class VendorBehaviorTest {
      public static void main(String[] args) {
          TicketPool ticketPool = new TicketPool();

          // FastVendor
          Thread fastVendor = new Thread(new FastVendor(ticketPool, 5));
          // SlowVendor
          Thread slowVendor = new Thread(new SlowVendor(ticketPool, 5));

          fastVendor.start();
          slowVendor.start();
      }
  }
  ```
- **5. Periodic Ticket Statistics Reporting**
  Task: Implement a thread to report ticket statistics every 5 seconds.
  Solution:
- Create a new thread to log total tickets added, removed, and currently in the pool.
- Use the Logger for periodic updates.
- **Test Case**: Run the system and confirm statistics are logged correctly at 5-second intervals.

- # 5. Periodic Ticket Statistics Reporting
  StatisticsReporter.java
- ```
  package threads;

  import core.TicketPool;
  import logging.Logger;

  public class StatisticsReporter implements Runnable {
      private final TicketPool ticketPool;

      public StatisticsReporter(TicketPool ticketPool) {
          this.ticketPool = ticketPool;
      }

      @Override
      public void run() {
          while (true) {
              try {
                  Logger.log("Total Tickets Added: X", "stats");
                  Logger.log("Total Tickets Removed: Y", "stats");
                  Logger.log("Current Tickets in Pool: " + ticketPool.getTicketCount(), "stats");
                  Thread.sleep(5000);
  ```

```
                } catch (InterruptedException e) {
                    Logger.log("Statistics reporting interrupted", "error");
                    break;
                }
            }
        }
    }
```

- **Statistics Reporting Test (Task 5: Periodic Statistics Reporting)**
- ```
  import core.TicketPool;
  import threads.StatisticsReporter;

  public class StatisticsReporterTest {
      public static void main(String[] args) {
          TicketPool ticketPool = new TicketPool();

          // Add tickets for testing
          ticketPool.addTickets("Ticket-1");
          ticketPool.addTickets("Ticket-2");

          // Start statistics reporting
          Thread statsReporter = new Thread(new StatisticsReporter(ticketPool));
          statsReporter.start();

          // Simulate adding and removing tickets
          ticketPool.removeTicket();
          ticketPool.addTickets("Ticket-3");

          try {
              Thread.sleep(10000); // Let stats run for a while
          } catch (InterruptedException e) {
              e.printStackTrace();
          }

          statsReporter.interrupt(); // Stop reporting
      }
  }
  ```
- **6. GUI for Dynamic Ticket Pool Visualization**
  Task: Enhance the GUI to show real-time ticket pool updates.
  Solution:
  - Add a ListView to the JavaFX GUI to display tickets in the pool.
  - Use Platform.runLater() to update the ListView dynamically as tickets are added or removed.
  - **Test Case**: Add and remove tickets during runtime and verify the ListView reflects real-time changes accurately.

6. GUI for Dynamic Ticket Pool Visualization

JavaFXInterface.java (Updated for ListView)

```
private ListView<String> ticketListView;

@Override
public void start(Stage primaryStage) {
    // Initialize GUI components as before
    ticketListView = new ListView<>();
    gridPane.add(new Label("Ticket Pool:"), 0, 8);
    gridPane.add(ticketListView, 0, 9, 2, 1);
}

private void updateTicketList() {
```
```

```
    Platform.runLater(() -> {
        ticketListView.getItems().clear();
        ticketListView.getItems().addAll(ticketPool.getTickets());
    });
}
```

To verify real-time updates for GUI Dynamic Visualization, you can create an automated test that simulates ticket additions and removals while checking the updates in the ListView. JavaFX has a TestFX library that simplifies GUI testing. Below is an example test case using TestFX.

Test Case for Real-Time Updates

This assumes you are using the TestFX library for testing JavaFX applications.

Step 1: Add TestFX Dependency If using Maven, add the following dependency to your pom.xml:

```
<dependency>
    <groupId>org.testfx</groupId>
    <artifactId>testfx-core</artifactId>
    <version>4.0.16-alpha</version>
</dependency>
<dependency>
    <groupId>org.testfx</groupId>
    <artifactId>testfx-junit5</artifactId>
    <version>4.0.16-alpha</version>
    <scope>test</scope>
</dependency>
```

Step 2: Test Case

```
import javafx.application.Platform;
import javafx.scene.control.ListView;
import javafx.stage.Stage;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.testfx.api.FxRobot;
import org.testfx.framework.junit5.ApplicationTest;
import ui.JavaFXInterface;
import static org.junit.jupiter.api.Assertions.assertEquals;
public class GUITest extends ApplicationTest {
private ListView<String> ticketListView;
@Override
    public void start(Stage stage) {
        JavaFXInterface app = new JavaFXInterface();
        app.start(stage);
        ticketListView = lookup("#ticketListView").queryAs(ListView.class); // ID must be set in
JavaFXInterface
    }
@BeforeEach
    public void setUp() {
        Platform.runLater(() -> ticketListView.getItems().clear());
    }
@Test
    public void testRealTimeUpdates() {
        Platform.runLater(() -> {
            ticketListView.getItems().add("Ticket-1");
            ticketListView.getItems().add("Ticket-2");
        });
// Wait for the JavaFX thread to complete
        FxRobot.waitForFxEvents();
```

```
// Verify the updates in the ListView
        assertEquals(2, ticketListView.getItems().size());
        assertEquals("Ticket-1", ticketListView.getItems().get(0));
        assertEquals("Ticket-2", ticketListView.getItems().get(1));
// Simulate a ticket removal
        Platform.runLater(() -> ticketListView.getItems().remove(0));
FxRobot.waitForFxEvents();
// Verify the ListView after removal
        assertEquals(1, ticketListView.getItems().size());
        assertEquals("Ticket-2", ticketListView.getItems().get(0));
    }
}
```

How It Works

1. **Setup**: The start method initializes the JavaFXInterface, and the ticketListView is retrieved by its ID (make sure to set the ID in the JavaFX code using setId).
2. **Test Actions**:
    - Tickets are added and removed using Platform.runLater(), ensuring updates happen on the JavaFX Application Thread.
    - The FxRobot.waitForFxEvents() method ensures all JavaFX events have been processed before assertions.
3. **Assertions**:
    - The ListView is checked for correct ticket additions and removals.

Setting the ID in JavaFXInterface

Update your JavaFXInterface to assign an ID to the ListView for testing:

```
ticketListView = new ListView<>();
ticketListView.setId("ticketListView"); // Add this line
```

Running the Test

1. Ensure the TestFX library is configured in your project.
2. Run the test using JUnit5 in your IDE or via Maven.

From <https://mail.google.com/mail/u/0/#search/oop%40iit.ac.lk/FMfcgzQXKhRczCKGrhtxzmGrkXKjhjfk>

new questions

Thursday, January 2, 2025 1:39 PM

I'll create 10 new questions and their solutions based on the provided ticket management system, focusing on different aspects of OOP, multi-threading, and GUI development.

```java
// QUESTION 1: Thread Pool Implementation for Multiple Vendors
// Implement a ThreadPoolExecutor to manage multiple vendor threads efficiently.
// Create a VendorPool class that uses the thread pool to handle vendor operations.

class VendorPool {
    private final ExecutorService executorService;
    private final TicketPool ticketPool;

    public VendorPool(int poolSize, TicketPool ticketPool) {
        this.executorService = Executors.newFixedThreadPool(poolSize);
        this.ticketPool = ticketPool;
    }

    public void startVendors(int vendorCount, int ticketReleaseRate) {
        for (int i = 0; i < vendorCount; i++) {
            executorService.submit(new Vendor(ticketPool, ticketReleaseRate));
        }
    }

    public void shutdown() {
        executorService.shutdown();
    }
}

// QUESTION 2: Ticket Expiration System
// Implement a ticket expiration mechanism where tickets become invalid after a certain time.

class ExpiringTicket {
    private final String id;
    private final LocalDateTime expirationTime;

    public ExpiringTicket(String id, Duration validityPeriod) {
        this.id = id;
        this.expirationTime = LocalDateTime.now().plus(validityPeriod);
    }

    public boolean isExpired() {
        return LocalDateTime.now().isAfter(expirationTime);
    }
}

class ExpiringTicketPool extends TicketPool {
```

```
    private final ScheduledExecutorService cleanupService =
        Executors.newSingleThreadScheduledExecutor();

    public ExpiringTicketPool() {
        cleanupService.scheduleAtFixedRate(
            this::removeExpiredTickets,
            0,
            1,
            TimeUnit.MINUTES
        );
    }

    private void removeExpiredTickets() {
        // Implementation to remove expired tickets
    }
}

// QUESTION 3: Observable Ticket Pool
// Implement the Observer pattern to notify subscribers of ticket pool changes.

interface TicketPoolObserver {
    void onTicketAdded(String ticket);
    void onTicketRemoved(String ticket);
}

class ObservableTicketPool extends TicketPool {
    private final List<TicketPoolObserver> observers = new CopyOnWriteArrayList<>();

    public void addObserver(TicketPoolObserver observer) {
        observers.add(observer);
    }

    @Override
    public synchronized void addTickets(String ticket) {
        super.addTickets(ticket);
        observers.forEach(o -> o.onTicketAdded(ticket));
    }

    @Override
    public synchronized String removeTicket() {
        String ticket = super.removeTicket();
        if (ticket != null) {
            observers.forEach(o -> o.onTicketRemoved(ticket));
        }
        return ticket;
    }
}

// QUESTION 4: Priority Queue Implementation
// Implement a priority-based ticket system using PriorityBlockingQueue.

class PriorityTicket implements Comparable<PriorityTicket> {
    private final String id;
```

```
    private final int priority;

    @Override
    public int compareTo(PriorityTicket other) {
        return Integer.compare(other.priority, this.priority);
    }
}

class PriorityTicketPool extends TicketPool {
    private final PriorityBlockingQueue<PriorityTicket> tickets =
        new PriorityBlockingQueue<>();

    // Implementation of priority-based add and remove operations
}

// QUESTION 5: Transaction History
// Implement a system to track and replay ticket operations.

class TicketTransaction {
    private final String ticketId;
    private final LocalDateTime timestamp;
    private final String operation;

    // Constructor and getters
}

class TransactionManager {
    private final Queue<TicketTransaction> transactions =
        new ConcurrentLinkedQueue<>();

    public void recordTransaction(TicketTransaction transaction) {
        transactions.offer(transaction);
    }

    public void replayTransactions(TicketPool pool) {
        // Implementation to replay transactions
    }
}

// QUESTION 6: Rate Limiting
// Implement rate limiting for ticket operations.

class RateLimitedTicketPool extends TicketPool {
    private final RateLimiter rateLimiter;

    public RateLimitedTicketPool(double operationsPerSecond) {
        this.rateLimiter = RateLimiter.create(operationsPerSecond);
    }

    @Override
    public synchronized void addTickets(String ticket) {
        rateLimiter.acquire();
        super.addTickets(ticket);
```

```
    }
}

// QUESTION 7: Ticket Categories
// Implement a category-based ticket system.

enum TicketCategory {
    VIP, STANDARD, ECONOMY
}

class CategorizedTicket {
    private final String id;
    private final TicketCategory category;

    // Constructor and getters
}

class CategoryTicketPool {
    private final Map<TicketCategory, TicketPool> pools =
        new EnumMap<>(TicketCategory.class);

    // Implementation of category-based operations
}

// QUESTION 8: Backup and Recovery
// Implement a backup system for ticket operations.

class TicketPoolBackup {
    private final Path backupPath;
    private final TicketPool pool;

    public void createBackup() throws IOException {
        // Implementation to save current state
    }

    public void restore() throws IOException {
        // Implementation to restore from backup
    }
}

// QUESTION 9: Custom Lock Implementation
// Implement a custom lock for ticket operations.

class TicketPoolLock {
    private final AtomicBoolean locked = new AtomicBoolean(false);
    private final Queue<Thread> waitingThreads = new ConcurrentLinkedQueue<>();

    public void lock() {
        Thread current = Thread.currentThread();
        waitingThreads.offer(current);

        while (waitingThreads.peek() != current ||
               !locked.compareAndSet(false, true)) {
```

```
            LockSupport.park();
        }

        waitingThreads.poll();
    }

    public void unlock() {
        locked.set(false);
        LockSupport.unpark(waitingThreads.peek());
    }
}

// QUESTION 10: Metrics Collection
// Implement a metrics collection system.

class TicketPoolMetrics {
    private final Map<String, AtomicLong> metrics = new ConcurrentHashMap<>();

    public void incrementMetric(String name) {
        metrics.computeIfAbsent(name, k -> new AtomicLong())
               .incrementAndGet();
    }

    public Map<String, Long> getMetrics() {
        return metrics.entrySet().stream()
            .collect(Collectors.toMap(
                Map.Entry::getKey,
                e -> e.getValue().get()
            ));
    }
}
```

Here's a brief description of each question and its solution:

1. Thread Pool Implementation: Creates a VendorPool class using ExecutorService to manage multiple vendor threads efficiently.

2. Ticket Expiration System: Implements an expiration mechanism where tickets become invalid after a set time period.

3. Observable Ticket Pool: Uses the Observer pattern to notify subscribers about ticket pool changes.

4. Priority Queue Implementation: Creates a priority-based ticket system using PriorityBlockingQueue.

5. Transaction History: Implements a system to track and replay ticket operations.

6. Rate Limiting: Adds rate limiting to control ticket operations per second.

7. Ticket Categories: Implements a category-based system (VIP, STANDARD, ECONOMY).

8. Backup and Recovery: Creates a backup system for ticket operations.

9. Custom Lock Implementation: Implements a custom lock for thread synchronization.

10. Metrics Collection: Implements a system to collect and report various metrics.

Each solution demonstrates different aspects of:
- Thread safety
- Design patterns
- Java concurrency utilities
- Error handling
- Performance optimization

Would you like me to explain any of these implementations in more detail?



ticket-syste
m-questio...

platform. run later

Monday, January 6, 2025 12:58 AM

Not exactly. Here's how the process works:

Initial UI Setup:

1. **JavaFX Application Thread**:
   - When you start a JavaFX application, the start() method runs on the JavaFX Application Thread.
   - This thread is responsible for creating and managing the UI components (like Stage, Scene, Label, etc.).
2. **UI Initialization**:
   - The UI components are initialized and displayed on the screen.
   - For example, in the start() method, you might set up the UI layout, add buttons, labels, and show the main application window (Stage).

Starting Background Threads:

1. **Background Threads**:
   - After setting up the UI, you can start background threads (like Vendor and Customer threads) to perform long-running or non-UI tasks (e.g., handling ticket operations in a ticketing system).
   - These threads run independently of the JavaFX Application Thread.
2. **Concurrent Execution**:
   - While the JavaFX Application Thread manages the UI, the background threads perform their tasks concurrently. They may handle business logic, data processing, or other operations that should not block the UI thread.

Updating UI from Background Threads:

1. **Platform.runLater**:
   - If a background thread needs to update the UI (e.g., updating a label with the number of tickets sold), it can't do so directly because UI updates must occur on the JavaFX Application Thread.
   - Instead, it uses Platform.runLater to queue a task that updates the UI.
   - The Runnable passed to Platform.runLater will be executed on the JavaFX Application Thread as soon as possible, after the current events in the JavaFX event queue are processed.

Key Points:

- **UI Setup First**: The UI components are initialized and shown first during the start() method execution.
- **Threads Run Concurrently**: Background threads start after the UI is set up and run concurrently with the JavaFX Application Thread.
- **UI Updates with Platform.runLater**: Any UI updates from these threads must use Platform.runLater to safely queue the updates on the JavaFX Application Thread.

Here's a simplified sequence:

1. JavaFX Application Thread starts and sets up the UI.
2. Background threads are started for tasks like ticket handling.
3. Background threads process tasks and use Platform.runLater to update the UI when necessary.
4. The JavaFX Application Thread continuously manages the UI and executes any queued tasks from Platform.runLater.

From <https://chatgpt.com/c/677ad4a4-6328-8008-a4f8-5ee2e3083354>

updated ticketPool

Tuesday, January 7, 2025 5:01 PM

```
package com.example.SpringBoot.model;

import com.example.SpringBoot.Service.LogService;
import org.springframework.beans.factory.annotation.Autowired;

import java.util.LinkedList;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
import java.util.logging.Logger;

public class TicketPool {
    private final Configuration config;
    private final LinkedList<Integer> ticketPool;
    private int numTicketForCustomers;
    private int numTicketByVendors;
    private int ticketNo = 1;

    private final Lock lock = new ReentrantLock();
    private final Condition poolNotFull = lock.newCondition();
    private final Condition poolNotEmpty = lock.newCondition();

    @Autowired
    private LogService logService;

    public TicketPool(Configuration config) {
        this.ticketPool = new LinkedList<>();
        this.numTicketForCustomers = config.getNoOfConsumer() * config.getTicketPerConsumer();
        this.numTicketByVendors = config.getNoOfVendor() * config.getTicketPerVendor();
        this.config = config;

        if (logService != null) {
            logService.addLog("TicketPool initialized with " + config.getMaxTicketCapacity());
        }
    }

    public void addTicket() throws InterruptedException {
        lock.lock();
        try {
            while (numTicketForCustomers == 0 || ticketPool.size() >= config.getMaxTicketCapacity()) {
                if (numTicketForCustomers == 0) {
                    logInfo("Vendor", Thread.currentThread().getName() + " - No consumers left to buy");
                    return;
                }
                logInfo("Vendor", Thread.currentThread().getName() + " Thread in wait. Pool is full");
                poolNotFull.await();
            }
```

```
            ticketPool.add(ticketNo++);
            numTicketByVendors--;

            logInfo("Vendor", Thread.currentThread().getName() + " added 1 ticket of ID: " + (ticketNo - 1) +
". Remaining tickets in pool: " + ticketPool.size());
            logService.addLog(Thread.currentThread().getName() + " added 1 ticket of ID: " + (ticketNo - 1) +
". Remaining tickets in pool: " + ticketPool.size() + ". (Total tickets sold: " + (ticketNo - 1) + ")");
            config.setTotalTicketSold(config.getTotalTicketSold() + 1);

            poolNotEmpty.signalAll();
        } finally {
            lock.unlock();
        }
    }

    public void removeTicket() throws InterruptedException {
        lock.lock();
        try {
            while (ticketPool.isEmpty()) {
                if (numTicketByVendors == 0) {
                    logInfo("Consumer", Thread.currentThread().getName() + " - cannot buy ticket. No tickets
left to sell");
                    return;
                }
                logInfo("Consumer", Thread.currentThread().getName() + " Thread in wait. Pool is empty");
                poolNotEmpty.await();
            }

            int ticket = ticketPool.removeFirst();
            numTicketForCustomers--;

            logInfo("Consumer", Thread.currentThread().getName() + " bought 1 ticket of ID: " + ticket + ".
Tickets remaining in pool: " + ticketPool.size());
            logService.addLog(Thread.currentThread().getName() + " bought 1 ticket of ID: " + ticket + ".
Tickets remaining in pool: " + ticketPool.size());

            poolNotFull.signalAll();
        } finally {
            lock.unlock();
        }
    }

    private void logInfo(String role, String message) {
        Logger.getLogger(role).info(message);
        logService.addLog(message);
    }
}
```